AccessRoute – AI-Powered Accessibility-Aware Navigation and Travel Planning System

A PROJECT REPORT

Submitted by

GEERTHANYAA G	**210425243067**
DEEBIGA S	**210425243046**

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CHENNAI INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

(Affiliated to Anna University, Chennai & Recognized by AICTE New Delhi)

Chennai-600 069

OCTOBER-2026

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

IM1.To create next generation leader by effective teaching learning methodologies and instill scientific spark in them to meet the global challenges.

IM2.To transform lives through deployment of emerging technology, novelty and sustainability.

IM3.To inculcate human values and ethical principles to cater the societal needs.

IM4.To contributes towards the research ecosystem by providing a suitable, effective platform for interaction between industry, academia and R&D establishments.

IM5.To nurture incubation centres enabling structured entrepreneurship and start-ups.

Vision of the Department:

To Achieve excellent standards of quality-education by using the latest tools, nurturing collaborative culture and disseminating customer oriented innovations to relevant areas of academia and industry towards serving the greater cause of society.

Mission of the Department:

IM1. To develop professionals who are skilled in the area of Artificial Intelligence and Data Science.

IM2.To impart quality and value based education and contribute towards the innovation of computing, expert system, Data Science to raise satisfaction level of all stakeholders.

.IM3. Our effort is to apply new advancements in high performance computing hardware and software.

CHENNAI INSTITUTE OF TECHNOLOGY (AUTONOMOUS)

(Affiliated to Anna University, Chennai & AICTE New Delhi)

Chennai-600 069

BONAFIDE CERTIFICATE

Certified that this project report "**AccessRoute – AI-Powered Accessibility-Aware Navigation and Travel Planning System**" is the bonafide work of **GEERTHANYAA G (210425243067)** and **DEEBIGA S (210425243046)** who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT
Department of Artificial Intelligence and Data Science,
Chennai Institute of Technology,
Kundrathur,
Chennai-600 069.

PROJECT SUPERVISOR
Dr.JASEEMA YASMIN
Assistant Professor,
Department of Artificial Intelligence and Data Science,
Chennai Institute of Technology,
Kundrathur,
Chennai-600 069.

Certified that the above students have attended a viva voce during the exam held on …….

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We express our gratitude to our Chairman Shri. P.SRIRAM and all trust members of Chennai Institute of Technology for providing the facility and opportunity to do this project as a part of our undergraduate course.

We are grateful to our Principal Dr. A.RAMESH M.E, Ph.D, for providing us the facility and encouragement during the course of our work.

We would like to extend our thanks to our Dean, Dr. V. SRINIVASA RAO, M.Tech, Ph.D, for providing valuable suggestions throughout this project.

We sincerely thank our Head of the Department, Department of Artificial Intelligence & Data Science, for having provided us with valuable guidance, resources and timely suggestions throughout our work.

We sincerely thank our Project Supervisor, **Dr.JASEEMA YASMIN**
Assistant Professor, Department of Artificial Intelligence & Data Science, for having provided us with valuable guidance, resources and timely suggestions throughout our work.

We would like to extend our thanks to our Project Coordinator, Dr. K.RAMANAN M.Tech, Ph.D, Associate Professor, Department of Artificial Intelligence & Data Science, for his valuable suggestions throughout this project.

We wish to extend our sincere thanks to all the Faculty Members of the Department of Artificial Intelligence & Data Science, for their valuable suggestions and their kind cooperation for the successful completion of our project.

We wish to acknowledge the help received from the Lab Instructors of the Department of Artificial Intelligence & Data Science, and others for providing valuable suggestions and for the successful completion of the project.

TABLE OF CONTENTS

NO.	TITLE	PAGE NO.
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	2
	1.3 ACCESSIBILITY BARRIERS IN URBAN NAVIGATION	2
	1.4 OBJECTIVES	3
	1.5 ROLE OF DATA SCIENCE AND AI IN ACCESSIBILITY-AWARE NAVIGATION	4
2	**LITERATURE SURVEY**	**6**
	2.1 EXISTING RESEARCH	6
	2.2 GAPS IN EXISTING KNOWLEDGE	8
	2.3 VALUE ADDITION OF THE PROPOSED SYSTEM	9
3	**PROBLEM DEFINITION AND REQUIREMENTS**	**10**
	3.1 EXISTING SYSTEM	10
	3.2 PROPOSED SYSTEM	10
	3.3 FUNCTIONAL REQUIREMENTS	12
	3.4 NON-FUNCTIONAL REQUIREMENTS	12
	3.5 HARDWARE REQUIREMENTS	13
4	**PROPOSED METHODOLOGY**	**14**
	4.1 OVERALL METHODOLOGY	14
	4.2 DATASET COLLECTION	14
	4.3 DATASET DESCRIPTION	15
	4.4 DATA PREPROCESSING	16
	4.5 EXPLORATORY DATA ANALYSIS	17
5	**SYSTEM DESIGN AND ARCHITECTURE**	**27**
	5.1 SYSTEM ARCHITECTURE	27
	5.2 DATA FLOW DIAGRAM	28
	5.3 USE CASE DIAGRAM	28
	5.4 SEQUENCE DIAGRAM	29
	5.5 DATABASE DESIGN	29
6	**IMPLEMENTATION**	**30**
	6.1 DEVELOPMENT ENVIRONMENT	30
	6.2 DATA PROCESSING	31
	6.3 FEATURE PROCESSING	31
	6.4 ALGORITHM IMPLEMENTATION	32
	6.5 REFERENCE IMPLEMENTATION AND EXECUTION	34
7	**RESULTS AND DISCUSSION**	**36**

7.1 EXPERIMENTAL SETUP....36
7.2 EVALUATION METRICS....36
7.3 ALGORITHM PERFORMANCE....37
7.4 COMPARATIVE ANALYSIS....38
7.5 RECOMMENDATION RESULTS....39
8 CONCLUSION AND FUTURE ENHANCEMENT....42
8.1 SUMMARY OF THE WORK....42
8.2 KEY CONTRIBUTIONS....42
8.3 LIMITATIONS....43
8.4 FUTURE ENHANCEMENTS....43
REFERENCES....44
APPENDIX....46
PO & PSO ATTAINMENT....47

ABSTRACT

Conventional navigation applications choose routes mainly by distance or travel time and assume that every traveller has the same needs. For a wheelchair user, a person with limited mobility, or a person with visual or hearing impairment, the shortest route may include stairs, steep slopes, blocked paths or inaccessible entrances. Accessibility information is also incomplete and uneven in quality, and applications rarely show how reliable it is. This project presents AccessRoute, an AI-powered accessibility-aware navigation and travel planning system developed as a Data Science PBL project.

AccessRoute lets users select one of five accessibility profiles and then adapts route evaluation, place discovery and travel planning to the profile. Each route and place is described by an explainable accessibility score derived from features such as step-free access, ramps, elevators, accessible restrooms, parking, obstacles and slopes, with profile-dependent weights. A data confidence model records the source, verification state and age of every attribute using the levels HIGH, MEDIUM, LOW and UNKNOWN, and missing attributes are kept as UNKNOWN instead of being assumed available. Routes are compared on distance, time, accessibility, obstacle risk and confidence, and a Pareto-based selection recommends a route according to the profile. Users can report obstacles, and the system re-evaluates the alternatives when a report affects a route.

The system also includes accessibility-aware discovery of hotels, restaurants, attractions and buildings, an AI Travel Planner and a hybrid chatbot in which Gemini works only on structured, location-specific context retrieved by the backend, while factual accessibility answers are produced deterministically. Real data and demonstration data are kept separate, and implemented components are separated from planned work.

The algorithms were validated on a reference implementation with synthetic data. All 25,884 property and regression checks passed. On 3,000 synthetic route requests the recommended route differed from the shortest route in 46 % to 86 % of the requests, depending on the profile, and between 87 % and 96 % of the recommendations were unchanged when the design weights were perturbed. These results describe the behaviour of the algorithms under stated assumptions; they are not measurements of accessibility in the real world.

Key Words: Accessibility-aware navigation; multi-objective routing; Pareto selection; data confidence; FastAPI; Mapbox; Gemini; travel planning.

LIST OF FIGURES

NO.	TITLE	PAGE NO.
6.1	Find Accessible Route and Live Accessibility Map	36
6.2	AI Travel Planner: Personalised Travel Plan	36
6.3	AccessRoute AI Hybrid Chatbot	36
7.1	Difference between the Recommended and the Shortest Route: (a) Share of Requests, (b) Mean Gain and Cost	38
7.2	Weight Sensitivity and Personalisation: (a) Recommendations Unchanged under Weight Noise, (b) Top-5 Overlap between Profiles	39

LIST OF TABLES

NO.	TITLE	PAGE NO.
1.1	Qualitative Importance of Accessibility Criteria by Profile (Design Assumption)	3
2.1	Related Work and the Contribution of AccessRoute	9
3.1	Implementation Status of AccessRoute Components	11
3.2	Functional Requirements of AccessRoute	12
3.3	Hardware Requirements (Recommended Configuration)	13
3.4	Software Requirements	13
4.1	Data Categories Used in AccessRoute	15
4.2	Composition of the Synthetic Reference Dataset	16
4.3	Preprocessing Techniques in AccessRoute	16
4.4	Illustrative Default Feature Weights $w_k(\pi)$ for Place Scoring	18
4.5	Illustrative Default Objective Weights $\omega_m(\pi)$ for Route Selection	19
4.6	Illustrative Source Reliability Weights ρ	21
4.7	Illustrative Candidate Routes (Example Values, Not Measured Data)	23
4.8	Parameter Values Used in the Reference Implementation	26
6.1	Division of Work Between Team Members	30
7.1	Property and Regression Tests of the Reference Implementation	37
7.2	Recommended Route versus Shortest Route on 3,000 Synthetic Requests	38
8.1	Limitations of the Present Work	43
A.1	Accessibility Attributes and Their Value Domains	46
PO.1	Program Outcome Attainment	47
PO.2	Program Specific Outcome Attainment	48

SYMBOLS & ABBREVIATIONS

Acronym	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ER	Entity-Relationship
GPS	Global Positioning System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
OSM	OpenStreetMap
PBL	Project Based Learning
REST	Representational State Transfer
UI	User Interface
UX	User Experience
UML	Unified Modeling Language
WCAG	Web Content Accessibility Guidelines
RAG	Retrieval-Augmented Generation
MOO	Multi-Objective Optimisation
UNKNOWN	Accessibility attribute whose value is not available

Symbols

Symbol	Meaning
$S(p, \pi)$	Accessibility score of place p for profile π (0–100)
$\kappa(p, \pi)$	Coverage: weight of the features whose value is known
S_{max}	Upper bound of the score if every UNKNOWN feature were available
$w_k(\pi)$	Weight of feature k for profile π
$\omega_m(\pi)$	Weight of objective m for profile π
ρ	Reliability of a data source
λ	Decay rate of data confidence per day

Symbol	Meaning
c_k, C	Confidence of attribute k and of a place
A(r), O(r)	Accessibility and obstacle risk of route r
μ	Decay rate of obstacle weight per day
ε	Obstacle buffer distance (m)
δ	Minimum utility improvement for rerouting
U(r, π)	Weighted utility of route r (lower is better)
α, β, γ	Weights of score, closeness and confidence in place ranking

CHAPTER 1
INTRODUCTION

Navigation technology has changed the way people move through cities, yet most of it was designed around an unstated assumption: that every traveller can use stairs, steep slopes, narrow doorways and uneven pavements. For a person who uses a wheelchair, a person with limited mobility, a person with low vision or a person who is deaf or hard of hearing, a route that is "shortest" or "fastest" may be unusable, unsafe or exhausting. This chapter introduces the background of the problem, states the problem formally, lists the objectives of the AccessRoute project, and defines the scope of the work. The organisation of the report is given at the end of the chapter.

1.1 BACKGROUND

Mobility is a precondition for education, employment, healthcare, social participation and leisure. When the physical environment and the information describing it are not accessible, a person's right to move independently is restricted even if the person is otherwise capable. Digital maps and navigation applications have become the default tool for finding routes and places, so the quality of accessibility information inside these tools directly affects the independence of a large group of users.

According to the World Health Organization, around 1.3 billion people, roughly one in six of the world's population, experience significant disability [3]. Ageing populations, chronic conditions and temporary injuries further increase the number of people who need accessible environments at some point in their lives. Universal design principles, accessibility legislation and digital accessibility standards such as the Web Content Accessibility Guidelines [4] have raised awareness of the issue, but the availability of reliable, machine-readable information about the accessibility of streets, buildings and facilities remains uneven across the world.

The 2011 Census of India recorded approximately 26.8 million persons with disabilities, about 2.21% of the population [10]. The Rights of Persons with Disabilities Act, 2016 recognises a wider list of disability categories than earlier legislation and places obligations on establishments and the government to provide accessible environments and services [11]. The Accessible India Campaign (Sugamya Bharat Abhiyan) was launched to improve accessibility of the built environment, transport and information technology [12].

1.2 PROBLEM STATEMENT

Existing mainstream navigation and travel-planning applications are optimised for speed and convenience for the average traveller. They typically provide a fastest or shortest route, sometimes a few alternatives, and a set of points of interest with ratings. Accessibility information, where it exists, is limited to a few labels (for example "wheelchair accessible entrance") that are shown without any indication of the source, age or reliability of the label. As a result, travellers with accessibility requirements must cross-check multiple sources, contact establishments individually, or accept risk.

The core problem addressed by this project can be stated as follows: *given an origin, a destination and a user's accessibility profile, how can a system compare several candidate routes and places using accessibility-related criteria, represent missing or uncertain accessibility information honestly, adapt to newly reported obstacles, and present the results in an explainable form together with AI-assisted travel planning, without inventing facts that are not present in its data?*

Four technical difficulties make the problem non-trivial. First, accessibility is multi-dimensional: no single number such as distance can capture it, and the importance of each dimension depends on the user's profile. Second, accessibility data is incomplete, comes from heterogeneous sources and has varying reliability, so the absence of a tag cannot be treated as the absence of a barrier or as the presence of a facility. Third, the situation on the ground changes, for example when a ramp is broken or a path is blocked, so static scoring is insufficient. Fourth, large language models are useful for explanation and planning but are known to produce fluent statements that are not grounded in facts [13], which is unacceptable when the statement concerns whether a hotel has an accessible room.

1.3 ACCESSIBILITY BARRIERS IN URBAN NAVIGATION

Barriers that affect travellers with accessibility requirements can be grouped into physical, informational and situational barriers.

Physical barriers include stairs without alternatives, steep slopes, narrow doors, high kerbs, lack of ramps, lifts that are absent or out of order, inaccessible toilets and the absence of accessible parking. **Informational barriers** include missing, outdated or unverified accessibility descriptions, and platforms that present a single "accessible" label without detail. **Situational barriers** include temporary obstacles such as construction, parked vehicles or flooding, which change quickly and are rarely recorded in static datasets.

The relative importance of these barriers differs from one profile to another. Table 1.1 illustrates, in qualitative terms, how the importance of different criteria varies. These qualitative levels motivate the profile-specific weight vectors introduced in Chapter 4; they are design inputs and not results of a user study.

Table 1.1 Qualitative Importance of Accessibility Criteria by Profile (Design Assumption)

Criterion	Wheelchair / mobility assistance	Limited mobility	Visual	Hearing	General
Step-free entrance and ramps	Very high	High	Medium	Low	Medium
Elevators / lifts	Very high	High	Medium	Low	Medium
Slopes and stairs on route	Very high	High	Medium	Low	Medium
Obstacles on route	High	High	Very high	Medium	Medium
Accessible restrooms	High	High	Medium	Medium	Medium
Accessible parking	High	Medium	Low	Low	Medium
Data confidence	High	High	High	High	High

1.4 OBJECTIVES

The primary aim of this project is to design and develop an accessibility-aware navigation and travel planning system that adapts routes, place recommendations and AI-generated plans to the accessibility profile of the user while being transparent about data quality.

1.4.1 General Objective

To build AccessRoute, a web-based system that goes beyond conventional route display by evaluating routes, places, facilities and travel plans against personalised accessibility requirements, and by explicitly modelling the uncertainty of accessibility data.

1.4.2 Specific Objectives

- To collect location, route and place data for selected regions using Mapbox services and open geographic data, and to keep real data, user-reported data and synthetic demonstration data separate and tagged by source.
- To clean, preprocess and integrate heterogeneous data, representing unavailable accessibility attributes as UNKNOWN rather than assuming availability.
- To compute an explainable, profile-weighted accessibility score for places and routes.

- To compare routes on multiple objectives (distance, time, accessibility, obstacle risk and confidence) and select non-dominated routes by Pareto-based selection.
- To compute data confidence from source, verification, age and corroboration, and to include it in route and place evaluation.
- To support user obstacle reporting and dynamic, obstacle-aware rerouting.
- To rank accessible hotels, restaurants, attractions and buildings for a given profile and location.
- To implement a FastAPI backend that orchestrates authentication, location processing, route requests, accessibility processing, scoring, recommendation, travel planning and chatbot communication.
- To integrate Gemini for itinerary generation, route explanation and conversational assistance, using only structured data retrieved by the backend.
- To implement a hybrid chatbot in which factual answers are produced deterministically by the backend and explanatory answers by Gemini.
- To provide a dashboard with a live map, route cards, route comparison, accessibility score, overview charts, nearby places, saved places, reports and ratings.
- To test the system against a documented set of functional, integration, usability and failure-handling test cases, and to record the outcome of each test case.

1.5 ROLE OF DATA SCIENCE AND AI IN ACCESSIBILITY-AWARE NAVIGATION

Data science contributes at three levels. At the data level, it provides methods for collecting, cleaning, integrating and quality-checking heterogeneous location and accessibility data [14, 15]. At the analytical level, it provides feature engineering, scoring, multi-criteria decision methods [16, 17] and multi-objective optimisation concepts such as Pareto dominance [18, 19]. At the presentation level, it provides visual analytics that turn numerical outputs into maps, cards and charts which users can interpret quickly.

Artificial intelligence, and specifically large language models, contribute natural-language understanding and generation. In AccessRoute this strength is used for itineraries, summaries and explanations. Language models are not used as a source of accessibility facts. Following the idea of grounding generation in retrieved evidence [20], the backend retrieves location-specific structured data first, and Gemini [21] is asked to write only from that context. The AI component is therefore a presentation and reasoning layer on top of data, and not a replacement for data.

1.6 SIGNIFICANCE

The significance of the project lies in the combination of ideas rather than in any single technique:

- Accessibility is treated as a multi-dimensional, profile-dependent property, not a single yes/no label.
- Uncertainty is made visible: UNKNOWN is a legitimate value, and confidence accompanies scores.
- Routes are compared on several objectives, and trade-offs between shortest distance and higher accessibility are shown to the user rather than hidden.
- User reports feed back into evaluation, so that the system can respond to obstacles.
- Generative AI is constrained to retrieved facts, which reduces the risk of fabricated accessibility claims.

1.7 SCOPE

The scope of the project is a web-based system with user and maintainer roles and five accessibility profiles, offering accessibility-aware route comparison and map visualisation, discovery of hotels, restaurants, attractions and buildings, explainable scoring with data confidence, obstacle reporting with rerouting, an AI Travel Planner, a hybrid chatbot and a dashboard.

The following are outside the scope of the current work and are discussed as future enhancements: nationwide data coverage, continuous real-time sensing of obstacles, multimodal public transport routing, offline navigation, multilingual interfaces, wearable integration and learned (machine-learning) accessibility prediction.

1.8 ORGANISATION OF THE REPORT

Chapter 2 surveys the literature and identifies the gaps. Chapter 3 defines the problem and the requirements. Chapter 4 presents the methodology: the dataset, preprocessing, feature engineering, the algorithms and their parameters. Chapter 5 describes the system design, Chapter 6 the implementation and the user interface, Chapter 7 the results, the comparison with a distance-only baseline and the application testing, and Chapter 8 concludes the report with the future enhancements.

CHAPTER 2
LITERATURE SURVEY

This chapter reviews the literature and existing practice that are relevant to AccessRoute. The review covers shortest-path routing, accessibility-aware routing and crowdsourced accessibility data, open geographic data, multi-objective and multi-criteria decision methods, data quality, and the use of retrieval-grounded generative AI. Only works that could be identified with confidence are cited, and the descriptions are limited to what the cited works are known to address. The chapter ends with the gaps that motivate AccessRoute and the value that the proposed system adds.

2.1 EXISTING RESEARCH

2.1.1 Shortest-Path Routing: Dijkstra's Algorithm and A*

Dijkstra described an algorithm for finding the shortest path between nodes of a graph with non-negative edge weights [1]. Hart, Nilsson and Raphael introduced the A* algorithm, which adds a heuristic estimate of the remaining cost to guide the search and is guaranteed to find a minimum-cost path when the heuristic is admissible [2]. These algorithms are the foundation of most road and pedestrian routing engines. Their output is optimal only with respect to the single cost function that is supplied. If the cost is distance or time, the route is optimal for distance or time; accessibility is respected only if it has been encoded into the edge weights or into constraints that exclude edges.

2.1.2 U-Access: Routing Pedestrians of Differing Abilities

Sobek and Miller presented U-Access, a web-based system that computes routes for pedestrians whose abilities differ, so that the route reflects the mobility constraints of the traveller [8]. The work demonstrates that it is feasible to attach ability-related attributes to a pedestrian network and to compute different routes for different users. It is an early example of treating accessibility as a property of the routing problem and not as an afterthought. The study depends on the availability of detailed network attributes, which is precisely the data that is difficult to obtain at scale.

2.1.3 Project Sidewalk: Crowdsourcing Sidewalk Accessibility Data

Saha et al. described Project Sidewalk, a web-based crowdsourcing tool in which contributors label accessibility problems and features, such as missing curb ramps or obstacles, by exploring street-level imagery [9]. The contribution of the work is a scalable way of collecting sidewalk accessibility data when authoritative datasets do not exist. It also

illustrates a recurring theme for accessibility data: crowd contributions can extend coverage, but the quality of individual labels varies, which creates a need for validation and for explicit treatment of reliability.

2.1.4 OpenStreetMap and User-Generated Geographic Information

Haklay and Weber described OpenStreetMap as a user-generated street map project and discussed how volunteers contribute geographic data [22]. The OpenStreetMap project documents tags, such as the wheelchair key, that allow contributors to record whether a feature can be used by wheelchair users [5]. The data is made available under an open licence [23]. For an accessibility-aware system, OpenStreetMap is a valuable source of real, openly licensed geographic and place information, but the presence, completeness and recency of accessibility tags differ by location, and a missing tag means "not recorded" rather than "not accessible".

2.1.5 Pareto Optimality and Multi-Objective Optimisation

When several objectives conflict, there is generally no single solution that is best on every objective. A solution is Pareto optimal (non-dominated) if no other solution is at least as good on all objectives and strictly better on at least one [18, 19]. Kung, Luccio and Preparata gave algorithms for finding the maxima of a set of vectors, which is the computational form of finding a Pareto front, with divide-and-conquer methods that are efficient in low dimensions [24]. Deb et al. proposed NSGA-II, a fast elitist non-dominated sorting genetic algorithm for multi-objective optimisation [25]. AccessRoute does not need an evolutionary algorithm because the candidate route set returned by a routing service is small; it uses the dominance concept directly on that set.

2.1.6 Data Quality in Data Science

Wang and Strong analysed data quality from the viewpoint of data consumers and grouped quality dimensions into intrinsic, contextual, representational and accessibility categories [15]. Wickham's work on tidy data describes a consistent structure for preparing datasets for analysis [14]. These works support the principle that data quality should be measured and carried along with the data. In accessibility navigation, accuracy, timeliness and provenance of the data are not abstract concerns because an error can lead a user to an inaccessible place.

2.1.7 Retrieval-Grounded Generation

Lewis et al. proposed retrieval-augmented generation, in which a language model conditions its output on documents retrieved from an external knowledge source, and showed benefits for knowledge-intensive tasks [20]. The AccessRoute AI components follow the same

architectural idea in a simplified form: the backend retrieves structured, location-specific data, and the model is asked to produce text only from that data.

2.1.8 Hallucination in Language Generation

Ji et al. surveyed hallucination in natural language generation, that is, generated content that is nonsensical or unfaithful to the source or to reality [13]. This literature explains why language models should not be used as a source of accessibility facts. If a model is asked "Is Hotel X wheelchair accessible?" without grounding, it may produce a confident but unsupported answer. AccessRoute therefore handles factual accessibility queries deterministically and restricts the model to explanation and planning over supplied data.

2.2 GAPS IN EXISTING KNOWLEDGE

From the review, the following gaps motivate the proposed system. They are gaps in the combination of capabilities and in handling of uncertainty and not claims that no individual capability exists elsewhere.

- **Profile-dependent weighting across several disability categories** in a single, configurable scoring framework that is applied to both routes and places.
- **Explicit representation of unknown accessibility information** and propagation of data confidence into route and place evaluation, rather than silent assumptions.
- **Presentation of trade-offs** through Pareto-based selection so that a shortest route that is less accessible is shown alongside a longer but more accessible one.
- **Closing the loop with obstacle reports**, so that user-reported obstacles influence the evaluation of routes in use and of future routes.
- **Unifying routing, place discovery and trip planning** in one system with common accessibility data.
- **Grounded AI assistance** where the language model is prevented from inventing accessibility facts.

Table 2.1 relates the reviewed works to these gaps. The "primary focus" column paraphrases what each cited work addresses; the last column states what AccessRoute adds.

Table 2.1 Related Work and the Contribution of AccessRoute

Work / approach	Primary focus	What AccessRoute adds
Dijkstra; A* [1, 2]	Optimal paths for a supplied cost function	Accessibility, obstacles and confidence in the evaluation of candidate routes
U-Access [8]	Routing for pedestrians of differing abilities	Multiple disability profiles, confidence, Pareto comparison, AI planning
Project Sidewalk [9]	Crowdsourced collection of sidewalk accessibility data	Use of reports in scoring and rerouting; verification by maintainers
OpenStreetMap wheelchair tags [5]	Open, community-maintained accessibility tags	UNKNOWN handling, source tracking, profile-based scoring
Wheelmap; AccessMap [6, 7]	Community accessibility mapping; pedestrian planning	Combined routing, place ranking and itinerary generation
Pareto and MCDA methods [24, 16]	General multi-objective and multi-criteria methods	Domain-specific objectives and profile weights for accessibility
RAG [20]	Grounded text generation	Application to accessibility planning with explicit "do not invent" rules

2.3 VALUE ADDITION OF THE PROPOSED SYSTEM

AccessRoute adds value by converting accessibility attributes into a documented numerical representation that preserves the distinction between "available", "not available" and "unknown"; by returning factor-wise contributions with every score so that the result is explainable; by reporting confidence alongside the score; by offering several non-dominated routes and recommending one according to the profile; by using user and maintainer inputs to keep the data current and to respond to obstacles; and by using Gemini only for explanation, summarisation and itinerary writing on top of retrieved facts. It is implemented as a complete web application with a dashboard, so that the algorithms can be used end to end. These points position AccessRoute as a decision-support system for accessible travel and not merely as a map viewer.

CHAPTER 3
PROBLEM DEFINITION AND REQUIREMENTS

3.1 EXISTING SYSTEM

Existing systems provide general-purpose routing, place search and itinerary tools. Some provide accessibility labels for places, and community projects collect accessibility data. As analysed in Chapter 1 and Chapter 2, they generally optimise a single objective, present coarse accessibility labels with little information about source and reliability, are not adapted to a selected accessibility profile, offer limited structured feedback for obstacles, and rely on separate tools for navigation, place discovery and planning.

3.2 PROPOSED SYSTEM

AccessRoute is a web-based, accessibility-aware navigation and travel planning system. A user registers, logs in and selects an accessibility profile. The user can then search for an origin and destination, view several route alternatives on a map, compare them using accessibility score, distance, time, obstacle risk and confidence, and receive one recommended route with an explanation. The user can also discover accessible hotels, restaurants, attractions and buildings, report obstacles, ask a chatbot, and generate a location-aware travel plan. Maintainers have a separate access level to verify data and moderate reports. The system architecture is described in detail in Chapter 5 and the implementation in Chapter 6.

The novelty of AccessRoute is the integration of the following elements into one pipeline: (i) personalised accessibility profiles; (ii) multi-objective route comparison; (iii) explainable accessibility scoring; (iv) data-confidence-aware navigation; (v) Pareto-based route selection; (vi) user obstacle reporting; (vii) dynamic obstacle-aware rerouting; (viii) accessibility-aware place discovery; and (ix) AI-assisted travel planning grounded in retrieved data. Each element is individually known in the literature or in practice; the contribution of this project lies in combining them, in the honest handling of unknown and low-confidence data, and in making the reasoning visible to the user. The report does not claim a new fundamental algorithm: the algorithmic contribution is the adaptation and composition of established ideas (weighted scoring, Pareto dominance, geometric obstacle checking and retrieval-grounded generation) to the accessibility navigation problem.

As required by the academic and ethical standards of the report, each component is labelled as implemented in the prototype or planned. Table 3.1 summarises the status that has been reported by the project team.

Table 3.1 Implementation Status of AccessRoute Components

Component	Status	Remarks
User authentication; user and maintainer roles	Implemented (prototype)	Role-based access is used to separate user and maintainer functions
Accessibility profiles (five categories)	Implemented (prototype)	Profile selection adapts weighting and recommendations
Geocoding, map display, route alternatives	Implemented (prototype)	Mapbox services
Accessibility scoring with factor breakdown	Implemented (prototype)	Profile-weighted; explanation returned with the score
Data confidence and source tracking	Implemented (prototype)	HIGH / MEDIUM / LOW / UNKNOWN levels
Multi-objective comparison; Pareto-based selection	Implemented (prototype)	Applied to the candidate route set
User obstacle reporting; obstacle-aware rerouting	Implemented (prototype)	Report-triggered re-evaluation; no continuous live sensing
Accessible place discovery (hotels, restaurants, attractions, buildings)	Implemented (prototype)	Location-specific retrieval
AI Travel Planner and hybrid chatbot (Gemini)	Implemented (prototype)	Gemini receives backend-retrieved structured context only
Dashboard and visualisations	Implemented (prototype)	Cards, charts, maps, hover details
Post-generation validation of AI output against retrieved entities	Planned	Designed as an additional safeguard
Large-scale crowd-sourced verification workflow	Planned	Future enhancement
Learned weights / predictive accessibility models	Planned	Future enhancement
Offline navigation; multilingual interface; multimodal transport	Planned	Future enhancement

3.3 FUNCTIONAL REQUIREMENTS

The functional requirements are derived from the objectives of Section 1.4 and the scope of Section 1.7. They are listed in Table 3.2 with the module that realises each of them.

Table 3.2 Functional Requirements of AccessRoute

ID	Requirement	Module (Section)
FR-01	Register users, authenticate them and separate user and maintainer roles	Authentication (6.4.1)
FR-02	Let the user select one of five accessibility profiles and apply the profile weights to every evaluation	Profile (6.3.1)
FR-03	Geocode origin and destination and retrieve several route alternatives	Location and Mapbox (6.2.1)
FR-04	Compute an explainable accessibility score with coverage, upper bound and factor breakdown	Scoring (6.3.2)
FR-05	Compute data confidence (HIGH, MEDIUM, LOW, UNKNOWN); never treat UNKNOWN as available	Scoring (6.3.2)
FR-06	Evaluate routes on five objectives, keep the Pareto set and recommend one route	Route evaluation (6.4.2)
FR-07	Accept obstacle reports and re-evaluate the alternatives	Obstacle and rerouting (6.4.3)
FR-08	Rank places by profile-weighted score, distance and confidence; separate verified from unverified places	Place discovery (6.4.4)
FR-09	Generate a location-aware travel plan from backend-retrieved context	AI Travel Planner (6.4.5)
FR-10	Answer factual questions deterministically and explanatory questions with Gemini on retrieved context	Hybrid chatbot (6.4.6)
FR-11	Present routes, scores, charts and reports on a dashboard with a live map	Dashboard (6.4.7)
FR-12	Allow maintainers to verify data and moderate reports	Authentication; data processing

3.4 NON-FUNCTIONAL REQUIREMENTS

The system must be secure, reliable (graceful handling of invalid locations, missing accessibility data and external service failures), transparent (every score is accompanied by a factor breakdown and a confidence level), maintainable (separation of routers, services and data access so that algorithms can be tested independently), usable and accessible (a readable, keyboard-operable dashboard that does not rely on colour alone) and privacy-preserving (only the data needed for the functions is stored, and API keys are never exposed to the browser).

3.5 HARDWARE REQUIREMENTS

AccessRoute is a web application and the backend performs light numerical computation (Section 7.3.2), so no special hardware is needed. Table 3.3 gives the recommended configuration.

Table 3.3 Hardware Requirements (Recommended Configuration)

Component	Server / development machine	Client device
Processor	Dual-core 2 GHz or better	Any current processor
Memory	8 GB RAM	4 GB RAM
Storage	10 GB free disk space	Not required
Network	Internet access to Mapbox and Gemini	Internet access
Machine used by the team	Core i5, 8 GB RAM, 512 GB SSD	Same laptop, Chrome

3.6 SOFTWARE REQUIREMENTS

The backend is written in Python and uses FastAPI with Pydantic [30, 31] for the REST interface, pandas and NumPy [26, 27] for data processing and Matplotlib [28] for charts. Mapbox [32] provides geocoding, route alternatives and map display, and the Gemini API [21] is used for the AI Travel Planner and the explanatory part of the chatbot. The frontend is a browser-based dashboard that communicates with the backend through JSON over HTTPS [33], and authentication tokens follow the JSON Web Token format [34]. Table 3.4 lists the software together with the versions used.

Table 3.4 Software Requirements

Purpose	Software	Version used
Programming language	Python	3.13.16
Backend framework	FastAPI with Pydantic	FastAPI 0.142.2, Pydantic 2.13.5
Data processing	pandas, NumPy, Matplotlib	pandas 3.0.6, NumPy 2.5.3, Matplotlib 3.11.2
Maps, geocoding, routing	Mapbox	Directions v5, Geocoding v6
Generative AI	Gemini API	Gemini API v1
Browser; version control	Chrome, Edge or Firefox; Git	Latest stable; Git 2.43.0

CHAPTER 4
PROPOSED METHODOLOGY

This chapter describes the methodology of AccessRoute: the overall system, the problem-solving approach based on a data science pipeline, the technologies used, the system architecture, the data handling, the algorithms for scoring and route selection, the AI Travel Planner and hybrid chatbot, and the output and visualisation strategy. Algorithms are presented using a consistent structure: concept, input, processing, algorithm, output and why it is useful. Throughout the chapter, components that are part of the implemented prototype are distinguished from components that are only planned.

4.1 OVERALL METHODOLOGY

The problem is treated as a data science pipeline with twelve stages. Each stage consumes the output of the previous stage and adds metadata (source, confidence, timestamp) so that provenance is never lost. The stages are:

1. **Data collection** – Mapbox location and route data, accessibility attributes, user and maintainer reports, and marked synthetic data (Section 4.2).
2. **Cleaning** – records are validated before they are used.
3. **Preprocessing** – raw records are converted to a consistent representation (Section 4.4).
4. **Integration** – data from the different sources is combined, with every record tagged with its source.
5. **Exploratory data analysis** – coverage and source distribution are summarised to show how much trust a score deserves (Section 4.5).
6. **Feature engineering** – attributes become numerical features such as suitability, coverage κ and confidence c (Section 4.6).
7. **Accessibility feature extraction** – place, barrier, route, segment, obstacle, confidence and coverage features are extracted from the integrated data.
8. **Scoring** – a profile-weighted accessibility score is computed for places and routes (Section 4.9.1).
9. **Route comparison** – each Mapbox alternative is evaluated on five objectives (Section 4.9.3).
10. **Recommendation** – Pareto selection keeps the non-dominated routes and one is recommended for the profile; places are ranked (Sections 4.9.4 and 4.9.7).

11. **Visualisation** – results are shown as map layers, route cards, scores, charts and text (Section 6.4.7).
12. **Evaluation** – experiments E0 to E6 are run on the synthetic reference dataset (Chapter 7).

4.2 DATASET COLLECTION

AccessRoute uses a combination of data categories, summarised in Table 4.1. The composition of the reference dataset that was used for the experiments of this report is described in Section 4.3.

Table 4.1 Data Categories Used in AccessRoute

Data category	Examples	Source type	Used for
Location and geocoding data	Place names, coordinates	Real (Mapbox)	Route requests, map display
Route data	Geometry, distance, estimated time, alternatives	Real (Mapbox)	Route comparison
Place and facility data	Hotels, restaurants, attractions, buildings	Real where available (open geographic data, place information)	Place discovery, planner context
Accessibility attributes	Ramps, lifts, step-free entrances, restrooms, parking, rooms, stairs, slopes, obstacles	Real where available; otherwise UNKNOWN	Scoring and filtering
User reports	Obstacle and accessibility reports	User-generated	Obstacle-aware evaluation, data updates
Verified project data	Maintainer-verified records	Maintainer	High-confidence data
Synthetic / demo data	Placeholder places and attributes	Synthetic (marked)	Development and testing only

Real and synthetic data are kept separate by a source-type attribute. Synthetic data exists only to allow development and testing when real accessibility data is missing or incomplete, and it is marked so that it can be excluded from real-world claims. Where accessibility information is unavailable, the attribute is stored as UNKNOWN and the system does not assume that the facility is available. Each attribute carries its source type, an optional reference, a retrieval or observation timestamp and a verification state; these fields feed the confidence calculation of Section 4.9.2. Place and facility retrieval is spatial: only records

around the geocoded destination (and, for the planner, along the corridor between origin and destination) are used, so that results from one location are never reused as if they described another.

4.3 DATASET DESCRIPTION

The experiments of this report use a **synthetic reference dataset** generated with a fixed random seed (42). It mimics the structure of the data that AccessRoute handles: places of four categories, nine accessibility attributes per place (Table A.1), and several observations per attribute from sources of different reliability and age. It is not real accessibility data and not the data store of the prototype; its purpose is to exercise the algorithms of Section 4.9 on an input whose composition is known. The generator assumptions are listed in Appendix B, and the composition of the dataset is given in Table 4.2.

Table 4.2 Composition of the Synthetic Reference Dataset

Property	Value
Places	500 (125 each of hotels, restaurants, attractions and buildings)
Attribute slots; observations	4,500 slots; 3,611 observations (0 to 3 per slot)
Observations by source	maintainer 7.4 %, real dataset 12.2 %, open map data 46.7 %, user report 19.3 %, synthetic 14.4 %
Reliability ρ of the sources	Maintainer 0.95, dataset 0.80, OSM 0.70, user 0.50, synthetic 0.20

4.4 DATA PREPROCESSING

Preprocessing converts raw records to a consistent representation. Table 4.3 lists the techniques.

Table 4.3 Preprocessing Techniques in AccessRoute

Task	Technique	Purpose
Coordinate validation	Range and format checks; rejection of null or out-of-range values	Prevent invalid map and route requests
Duplicate handling	Match on name, category and proximity	Avoid duplicate places
Category normalisation	Mapping of source tags to a fixed category list (hotel, restaurant, attraction, building)	Consistent filtering
Attribute normalisation	Mapping of values such as yes, no, limited, designated to a common set	Comparable accessibility attributes
Missing-value encoding	Missing or unrecognised attribute is stored as UNKNOWN	Avoid assuming availability
Text cleaning	Trimming, case and whitespace normalisation	Clean display and matching
Timestamp standardisation	Common date-time format	Age-based confidence
Source tagging	source_type, reference and timestamp attached to every record	Provenance and confidence

4.5 EXPLORATORY DATA ANALYSIS

Before scoring, the integrated data is explored to understand its quality [35]. Typical questions are: what fraction of places have each accessibility attribute known, how the sources are distributed (real, user, maintainer, synthetic), which categories have the weakest coverage, and where reports cluster. The result is a coverage summary that informs how much trust to place on scores in a region. The summary computed for the reference dataset is given below.

The coverage summary of the reference dataset, computed over all 4,500 attribute slots, is as follows:

- 44.9 % of the slots have no observation and are UNKNOWN.
- The remaining slots are LOW confidence (23.9 %), MEDIUM (21.3 %) and HIGH (9.9 %).
- Conflicting observations occur in 4.3 % of the slots.

The unknown rate follows from the generator setting (45 % of the slots receive no observation), chosen to reflect the sparse coverage described for real accessibility data; it is not a finding about real places.

The mean coverage κ of a place (Equation (4.2)) is 55.7 % for Wheelchair, 55.4 % for Limited mobility, 54.5 % for Visual, 55.2 % for Hearing, 55.1 % for General. Roughly half of the weight of each profile is therefore backed by evidence, and the rest is carried by the upper bound S_{max} rather than by assumed values.

4.6 FEATURE ENGINEERING

Feature engineering transforms geographic and accessibility attributes into numerical features that the algorithms can use. The feature groups are the place accessibility features (step-free entrance, ramp, elevator, ground-floor access, restroom, parking, room), barrier features (stairs, steep slope, obstacle), route features (distance, estimated time, segment length), segment features (slope, kerb, obstacle presence), obstacle features (type, severity, distance to route, age), confidence features (source reliability, verification, age, corroboration) and coverage features.

Three engineered quantities are especially important. The **suitability value** $s_k \in \{0, 1\}$ converts each attribute into a form in which larger is always better (a barrier that is absent is encoded as 1). The **coverage** κ measures how much of the weighted evidence is actually known. The **confidence** $c \in [0, 1]$ expresses how much the known evidence can be trusted. Together, these quantities let the system distinguish a place that is known to lack a ramp from a place for which nothing is known.

4.7 FEATURE SELECTION AND PROFILE WEIGHTING

AccessRoute does not learn which features matter. Feature selection is done by the **weight vector of the selected profile**: a feature that matters little to a profile receives a small weight and a critical feature a large one. The weights are design values that encode the qualitative importance of Table 1.1; they are not fitted to data.

Table 4.4 Illustrative Default Feature Weights $w_k(\pi)$ for Place Scoring

Feature	Wheelchair	Limited mobility	Visual	Hearing	General
Step-free entrance	0.20	0.15	0.08	0.12	0.111
Ramp	0.12	0.08	0.04	0.08	0.111
Elevator	0.15	0.18	0.08	0.10	0.111
Ground-floor access	0.08	0.12	0.08	0.10	0.111
Accessible restroom	0.15	0.12	0.10	0.14	0.111
Accessible parking	0.08	0.12	0.04	0.10	0.111
Accessible room	0.10	0.08	0.08	0.14	0.111
No obstacles nearby	0.06	0.07	0.30	0.12	0.111
No stairs	0.06	0.08	0.20	0.10	0.111
Sum	1.00	1.00	1.00	1.00	1.00

Table 4.5 Illustrative Default Objective Weights $\omega_m(\pi)$ for Route Selection

Objective	Wheelchair	Limited mobility	Visual	Hearing	General
Distance D	0.10	0.20	0.10	0.20	0.25
Travel time T	0.10	0.15	0.10	0.20	0.25
Inaccessibility $(100 - A)$	0.40	0.30	0.25	0.25	0.20
Obstacle risk O	0.25	0.20	0.35	0.20	0.15
Low confidence $(1 - C)$	0.15	0.15	0.20	0.15	0.15
Sum	1.00	1.00	1.00	1.00	1.00

The weights are design values that express the qualitative importance of Table 1.1. They have not been derived from a user study; a formal elicitation, for example using pairwise comparisons [17] with users who have the respective requirements, is proposed as future work. For the hearing profile, features such as visual alarms or captioned communication are not part of the present attribute set and are listed as future data attributes.

4.8 DATA SPLITTING AND EVALUATION PROTOCOL

A machine-learning model is trained on one part of the data and tested on another. AccessRoute has no parameters that are learned from data (Section 4.10), so there is no training set to separate from a test set and no accuracy against a ground-truth label can be

reported. The evaluation is therefore built around the questions the system must answer correctly. Seven independent experiments, each drawing its input from the generator with the common seed, were run: a description of the dataset (E0); 22 property and regression tests with 25,884 checks (E1); a benchmark of 3,000 route requests, 600 per profile, against a distance-only baseline (E2); a weight-sensitivity test on every benchmark request (E3); a personalisation test on 300 candidate sets of 40 places (E4); a rerouting simulation with 2,000 requests (E5); and a run-time study (E6).

4.9 ALGORITHMS AND ANALYTICAL MODELS

This section is the algorithmic core of the project. Eight components are described: accessibility scoring, personalised weighting, data confidence, multi-objective route evaluation, Pareto-based selection, obstacle-aware evaluation, dynamic rerouting and recommendation ranking. For each one the description follows the order concept, input, processing, algorithm, output and usefulness. The formulations are the design of the AccessRoute scoring framework; parameter values given in the tables are default values used to explain the method and are configurable. The numerical examples in this section are constructed for explanation; measured behaviour of the same algorithms on a synthetic reference dataset is reported in Chapter 7.

4.9.1 Accessibility Scoring Algorithm

The accessibility score summarises how well a place (or a route segment) satisfies the accessibility requirements of the selected profile. It is a weighted combination of feature suitability values, so that every point of the score can be traced to named features. Unknown attributes are not assumed to be available.

Each known feature contributes $w_k \cdot s_k$. An UNKNOWN feature contributes 0 to the score (conservative treatment) but is recorded in the explanation. The coverage κ is computed so that the user can see how much of the score is based on evidence.

$$S(p, \pi) = 100 \cdot \Sigma_k \, w_k(\pi) \cdot s_k(p) \tag{4.1}$$

$$\kappa(p, \pi) = \Sigma_{k\ known} \, w_k(\pi) \tag{4.2}$$

$$S_{max}(p, \pi) = S(p, \pi) + 100 \cdot (1 - \kappa(p, \pi)) \tag{4.3}$$

Equation (4.1) gives the evidence-based score, Eq. (4.2) the coverage and Eq. (4.3) an upper bound that would be reached if every unknown feature turned out to be available. The pair $[S, S_{max}]$ tells the user the range in which the true score lies given the present knowledge. A place with many unknown attributes therefore has a wide interval and cannot be confused with a place that is verified.

Algorithm 4.1 Accessibility score with factor breakdown

```
Input : place p, profile π, weights w, attribute table
A(p)
Output: score S, coverage κ, upper bound S_max, breakdown
B
S ← 0 ; κ ← 0 ; B ← empty list
for each feature k in K:
    v ← A(p, k)                          # yes / no / UNKNOWN
    if v = UNKNOWN:
        append (k, "unknown", 0, w[k]) to B
    else:
        s ← suitability(k, v)            # 1 if beneficial, 0
otherwise
        S ← S + w[k] * s ; κ ← κ + w[k]
        append (k, v, w[k] * s, w[k]) to B
S ← 100 * S ; S_max ← S + 100 * (1 − κ)
return S, κ, S_max, B
```

Consider a hotel for the wheelchair profile where only the step-free entrance, ramp and accessible parking are known and available, and all other features are UNKNOWN. With the weights of Table 4.4, $S = 100 \cdot (0.20 + 0.12 + 0.08) = 40$, $\kappa = 0.40$ and $S_{max} = 100$. The interface would show "40 (up to 100), coverage 40%", making clear that the hotel is not verified as inaccessible, nor as fully accessible.

4.9.2 Data Confidence Calculation

Confidence expresses how far the recorded value of an attribute can be trusted. It depends on the source, whether the record was verified, how old it is and whether independent observations agree.

A single observation has confidence $\rho_j \cdot \exp(-\lambda \cdot a_j)$. If several independent observations agree, their confidences are combined with a noisy-OR rule, so that agreement increases confidence but never beyond 1. If observations disagree, the system keeps the observation with the highest confidence, flags the conflict and lowers the displayed level. If there is no observation the attribute is UNKNOWN and its confidence is 0.

$$c_k = 1 - \Pi_j \left(1 - \rho_j \cdot exp(-\lambda \cdot a_j) \right) \tag{4.4}$$

$$C(p, \pi) = \Sigma_k \, w_k(\pi) \cdot c_k \tag{4.5}$$

Equation (4.4) gives the confidence of one attribute and Eq. (4.5) the weighted confidence of a place. The numeric confidence is mapped to a level: HIGH if $c \geq 0.75$, MEDIUM if $0.50 \leq c < 0.75$, LOW if $0 < c < 0.50$ and UNKNOWN if there is no observation.

Table 4.6 Illustrative Source Reliability Weights ρ

Source type	ρ	Remarks
Maintainer-verified record	0.95	Verified by a maintainer account
Real dataset / official source	0.80	Where available
Open map data (community tag)	0.70	Completeness and recency vary
User report	0.50	May be corroborated by other reports
Synthetic / demo data	0.20	For testing only; not used for real-world claims

4.9.3 Multi-Objective Route Evaluation

Routes are compared on several objectives and not only on distance. The objectives are distance D, estimated travel time T, accessibility A, obstacle risk O and route confidence C. Route accessibility is computed as the length-weighted mean of segment accessibility values $a_i \in [0, 100]$, where a_i is computed with Eq. (4.1) on segment features (step-free, slope, ramp or kerb, obstacle). A route that contains a segment with a known blocking barrier for the profile is marked infeasible and its accessibility is set to 0.

$$A(r) = (1 - H(r)) \cdot \Sigma_i \ell_i \cdot a_i / \Sigma_i \ell_i \tag{4.6}$$

$$C(r) = \Sigma_i \ell_i \cdot c_i / \Sigma_i \ell_i \tag{4.7}$$

Here ℓ_i is the length of segment i, c_i its confidence, and H(r) = 1 if the route contains a known blocking barrier for the profile and 0 otherwise. The obstacle risk O(r) is defined in Section 4.9.5. All objectives are converted to a "smaller is better" form, f(r) = (D, T, 100 − A, O, 1 − C), and normalised by min–max scaling over the candidate set, so that each normalised objective lies in [0, 1].

$$\tilde{f}_m(r) = (f_m(r) - \min_q f_m(q)) / (\max_q f_m(q) - \min_q f_m(q)) \tag{4.8}$$

4.9.4 Pareto-Based Route Selection

A route r_i dominates a route r_j if r_i is at least as good as r_j on every objective and strictly better on at least one. A route that is not dominated by any other is Pareto optimal [18]. Dominated routes are never better than some other route in any respect and can be discarded without asking the user for weights.

$$r_i \succ r_j \Leftrightarrow \forall\, m: f_m(r_i) \leq f_m(r_j) \wedge \exists\, m: f_m(r_i) < f_m(r_j) \tag{4.9}$$

$$U(r, \pi) = \Sigma_m \omega_m(\pi) \cdot \tilde{f}_m(r), \quad r^* = \text{argmin over } r \text{ in } P \text{ of } U(r, \pi) \tag{4.10}$$

Equation (4.9) defines dominance. Let P be the set of non-dominated routes. The recommended route r* is the member of P with the smallest weighted utility U, Eq. (4.10), for the selected profile.

Algorithm 4.2 Pareto filter and profile-based recommendation

```
Input : objective vectors F (normalised), profile weights
ω
Output: Pareto set P, recommended route r*, ranking of P
P ← empty
for each route a in R:
    dominated ← false
    for each route b in R, b ≠ a:
        if all(F[b][m] ≤ F[a][m]) and any(F[b][m] <
F[a][m]):
            dominated ← true ; break
    if not dominated: add a to P
for each route r in P: U[r] ← Σ_m ω[m] * F[r][m]
r* ← argmin U over P ; ranking ← P sorted by U
return P, r*, ranking
```

The simple pairwise filter costs $O(n^2 \cdot m)$ for n routes and m objectives. Since a routing service returns only a small number of alternatives, this is negligible. For large candidate sets, divide-and-conquer algorithms for the maxima of vectors are more efficient in low dimensions [24], and evolutionary multi-objective algorithms such as NSGA-II [25] would be an option if the problem were extended to generating routes instead of selecting them; both are beyond the present scope.

Table 4.7 lists five hypothetical candidate routes. Route A is the shortest but has low accessibility and a high obstacle risk. Route B is slightly longer but considerably more accessible. Route C is longer still, but has high confidence and the lowest obstacle risk. Routes D and E are dominated: D by B, and E by B and C. The Pareto set is P = {A, B, C}.

Table 4.7 Illustrative Candidate Routes (Example Values, Not Measured Data)

Route	Distance (km)	Time (min)	Accessibility A	Obstacle risk O	Confidence C	Pareto status
A	4.0	12	55	0.30	0.60	Non-dominated (shortest)
B	4.8	15	88	0.10	0.70	Non-dominated (most accessible)
C	5.6	18	82	0.02	0.90	Non-dominated (high confidence, low risk)
D	5.2	17	60	0.25	0.55	Dominated by B
E	6.0	20	70	0.15	0.65	Dominated by B and C

After min–max normalisation and application of the weights of Table 4.5, the recommended route changes with the profile: the same candidate set yields route C for the wheelchair and visual profiles, where obstacle risk and confidence are heavily weighted, and route B for the limited-mobility, hearing and general profiles. For the wheelchair profile, the difference between B (0.235) and C (0.228) is small, which the interface can express by presenting B as a close alternative.

4.9.5 Obstacle-Aware Route Evaluation

An obstacle is a reported or recorded condition at a location, such as a broken ramp, blocked path, unavailable elevator, construction or an inaccessible entrance, that reduces the usability of the segments near it. Obstacle risk summarises the effect of active obstacles on a route.

An obstacle affects a route if its distance to the polyline is at most ε. Distances are computed with the haversine formula [29] after a bounding-box prefilter. Each affecting obstacle contributes a probability-like weight $\rho_o \cdot \exp(-\mu \cdot \text{age})$, and the contributions are combined so that more obstacles increase the risk but never beyond 1. Obstacles that are verified by a maintainer keep full weight; unverified reports lose weight as they age unless they are corroborated. The severity weights ρo are 0.20 for low, 0.50 for medium and 0.90 for high severity; a low-severity obstacle is a minor temporary obstruction that can be bypassed, a

medium one a partially blocked path or damaged ramp, and a high one an unavailable elevator, a step-only entrance or a fully blocked path.

$$O(r) = 1 - \Pi_{o \in Ob(r)} (1 - \rho_o \cdot exp(-\mu \cdot age_o)) \quad (4.11)$$

4.9.6 Dynamic Rerouting

When a user reports an obstacle, or when an obstacle is added or verified by a maintainer, the route that the user is following may no longer be suitable. Dynamic rerouting is the process of detecting this and recommending a better alternative. In the present system this is a report-triggered re-evaluation; it is not continuous real-time sensing of the environment.

4.9.7 Recommendation Ranking

Hotels, restaurants, attractions and buildings are ranked for the profile and the location. The ranking uses the accessibility score, the proximity to the destination and the confidence, and applies must-have constraints of the profile.

The input is the candidate place set within the region of interest, the profile, the user's preferences and the weights α, β, γ. Places that violate a known must-have constraint (for example, a known absence of step-free entrance for the wheelchair profile) are excluded. Places for which a must-have attribute is UNKNOWN are not hidden; they are placed in a separate "unverified" group, because hiding them would remove potentially suitable options and showing them as accessible would be misleading.

$$R(p, \pi) = \alpha \cdot S(p, \pi)/100 + \beta \cdot (1 - d(p)/d_{max}) + \gamma \cdot C(p, \pi), \quad \alpha + \beta + \gamma = 1 \quad (4.12)$$

Here d(p) is the distance of the place from the point of interest and d_{max} the largest distance in the candidate set. Typical default weights are $\alpha = 0.6$, $\beta = 0.2$ and $\gamma = 0.2$.

4.9.8 AI Travel Planner and Hybrid Chatbot

The AI Travel Planner generates a location-aware, personalised travel plan. It accepts the origin, destination, travel dates, number of travellers, accessibility profile and accessibility preferences. It can generate a day-wise itinerary, accessible hotel suggestions, accessible restaurants, attractions, route suggestions, accessibility considerations, travel tips and route explanations.

Because the context is built by spatial retrieval, a plan for Chennai to Mahabalipuram uses different places and routes from a plan for Bengaluru to Mysuru. The AI model is not asked to recall hotels; it is asked to arrange retrieved ones.

The chatbot is hybrid: an intent router in the backend decides whether a message needs a deterministic answer or a generated explanation. Questions about routes, facilities, accessibility attributes, datasets and locations are answered directly by backend functions

from stored data; questions that ask for explanation, summarisation or natural-language planning are passed to Gemini together with the structured context retrieved by the backend. A hybrid design is preferable to letting a language model answer everything, because language models can produce fluent but unfaithful text [13] and because the cost of an inaccurate statement about accessibility is high. The measures are: (i) factual accessibility information is returned directly from stored attributes with source and confidence; (ii) Gemini receives only retrieved structured data, in the spirit of retrieval-grounded generation [20]; (iii) instructions require the model to say "not verified" for UNKNOWN attributes; (iv) confidence levels shown to the user are taken from the backend, not generated by the model; and (v) the planned post-generation check compares named entities with the context. These measures reduce, but do not eliminate, the risk of hallucination, so the report lists it as a limitation (Chapter 8) and recommends human verification of accessibility information.

4.10 PARAMETER SETTING

In a machine-learning project this stage is model training. AccessRoute has no training stage because none of its parameters is estimated from data: the weights are design values (Section 4.7) and the remaining constants are set from a stated meaning. Table 4.8 lists the values used by the reference implementation; all are configurable in the system.

Table 4.8 Parameter Values Used in the Reference Implementation

Parameter	Symbol	Value	Meaning
Confidence decay	λ	0.00190 per day	Half-life of 365 days
Obstacle freshness decay	μ	0.0231 per day	Half-life of 30 days for unverified reports
Obstacle buffer	ε	50 m	Distance within which an obstacle affects a route
Rerouting margin	δ	0.02	Minimum utility improvement before the route changes
Ranking weights	α, β, γ	0.6, 0.2, 0.2	Score, closeness and confidence (Equation (4.12))
Confidence thresholds	–	0.75 and 0.50	HIGH and MEDIUM levels
Reliabilities; severities	ρ, ρ_o	Table 4.6; Section 4.9.5	Source and obstacle-severity weights

4.11 SENSITIVITY ANALYSIS OF PARAMETERS

Hyperparameter tuning would search for the values that give the best validation score. AccessRoute has no ground-truth score to optimise, so the parameters are not tuned; the weights are instead **stress-tested**. Each objective weight vector ω is multiplied component-wise by $\exp(N(0, \sigma^2))$, renormalised to sum to 1, and the recommendation is recomputed, 20 times for every benchmark request at $\sigma = 0.1$ and $\sigma = 0.2$. A recommendation is stable if it does not change. The result (Section 7.4.2) shows how much the choice depends on the exact weights, and therefore how much they need to be refined by an elicitation with real users, which is proposed as future work.

CHAPTER 5
SYSTEM DESIGN AND ARCHITECTURE

This chapter presents the design of AccessRoute: the layered architecture, the data flow, the use cases, the activities and message sequences of a route request, the class model and the database design.

5.1 SYSTEM ARCHITECTURE

AccessRoute is a layered web application. A browser-based frontend communicates with a FastAPI backend over HTTPS. The backend authenticates the user, receives the selected accessibility profile, calls Mapbox for geocoding and route generation, retrieves accessibility data from its data store and open data sources, runs the scoring and selection algorithms, and, when required, builds a structured context for Gemini. The response is returned to the frontend as structured JSON and is rendered as map layers, route cards, scores, charts and text.

The architecture is organised into the following parts:

- **Data sources** – Mapbox and open geographic data, accessibility attributes, user and maintainer reports, and marked synthetic data.
- **Data ingestion and storage** – records are validated, tagged with their source and stored.
- **Processing and analytics** – features are extracted and confidence and scores are computed.
- **Routing and recommendation engine** – annotates the Mapbox alternatives with segment-level accessibility, evaluates the five objectives, applies Pareto selection and recomputes it when an obstacle is reported (the algorithms of Section 4.9).
- **Presentation layer** – the dashboard presents the results.
- **Integration workflow** – the FastAPI application is the integration point: its routers receive requests from the browser and call services, and the services access the data store, Mapbox, open data and Gemini.

5.2 DATA FLOW DIAGRAM

At Level 1 the process is decomposed into the following parts, with the principal data stores:

- **Authentication** – the backend authenticates the user.
- **Location and routing** – geocoding and route alternatives are obtained from Mapbox.

- **Accessibility processing** – accessibility data is retrieved and confidence and scores are computed.
- **Recommendation** – Pareto selection and ranking according to the selected profile.
- **Reporting and rerouting** – user obstacle reports are recorded and the alternatives are re-evaluated.
- **AI assistance** – the AI Travel Planner and the chatbot use Gemini on context retrieved by the backend.
- **Data stores** – accessibility attributes with their source and confidence, reports, travel plans and chat sessions.

5.3 USE CASE DIAGRAM

Three actors interact with the system:

- **User** – registers, logs in, selects a profile, searches routes, compares alternatives, discovers places, reports obstacles, plans trips and chats with the assistant.
- **Maintainer** – reviews and verifies reports and data.
- **External services (Mapbox and Gemini)** – supply geocoding and routes, and generated explanations.

5.4 SEQUENCE DIAGRAM

For one route request, the messages pass in the following order:

1. The browser sends the route request with the selected accessibility profile to the FastAPI backend.
2. The backend authenticates the user and receives the profile.
3. The backend calls Mapbox for geocoding and route alternatives.
4. The backend retrieves accessibility data from the data store and open data sources.
5. The scoring logic evaluates the five objectives and applies Pareto selection.
6. The response is returned to the browser as structured JSON.
7. The browser renders it as map layers, route cards, scores, charts and text.

5.5 DATABASE DESIGN

The data model contains the following entities:

- **User and AccessibilityProfile** – identity, role and the selected profile with its preferences.

- **Place, Facility and AccessibilityFeature** – the place, its facilities and each attribute with its value yes, no or UNKNOWN, confidence, source and verification.
- **Route, RouteAlternative and Obstacle** – the routes, their alternatives and the reported obstacles.
- **UserReport, TravelPlan, Itinerary, ChatSession and DataSource** – the reports, plans, chat sessions and data sources.

The attribute names must be aligned with the real schema by the backend developer.

5.6 DEPLOYMENT VIEW

The prototype is deployed as three components: the FastAPI application and the data store on the application server, the React dashboard served as static files, and the external services (Mapbox for geocoding and routes, Gemini for text generation) reached over HTTPS. The API key of each service is held in the server environment and is never sent to the browser. A production deployment has not been evaluated in this work.

CHAPTER 6
IMPLEMENTATION

This chapter describes how the design of Chapter 5 is realised as a working prototype: the development environment, the modules, the reference implementation that was used to validate the algorithms, deployment and the user interface. Components described as implemented belong to the prototype developed by the project team; components that are only designed are marked as planned.

6.1 DEVELOPMENT ENVIRONMENT

AccessRoute is implemented as a client–server web application. The backend, developed by Geerthanyaa G, is a FastAPI service responsible for authentication, location processing, route requests to Mapbox, accessibility data processing, scoring, recommendation, obstacle handling, travel planning and chatbot communication. The frontend, developed by Deebiga S, is a browser-based dashboard that provides the interface for login, profile selection, route search, map display, route comparison, place discovery, report submission, the AI Travel Planner and the chatbot.

Table 6.1 Division of Work Between Team Members

Area	Geerthanyaa G	Deebiga S
Authentication and roles	Token-based authentication, role checks, protected endpoints	Login and registration screens, session handling in the browser
Accessibility logic	Scoring, weighting, confidence, Pareto selection, obstacle risk	Display of scores, factor breakdown and confidence badges
Routing and maps	Mapbox requests, route normalisation, alternatives	Map rendering, route cards, route comparison view
Data processing	Cleaning, integration, source tagging, real/synthetic separation	Filters, category views, hover details
AI components	Structured context assembly, Gemini calls, hybrid chatbot routing	Travel Planner form, itinerary view, chat interface
Dashboard	Data endpoints for cards and charts	Layout, visual design, interaction, responsiveness

6.2 DATA PROCESSING

6.2.1 Location and Mapbox Module

This module converts place names to coordinates through the Mapbox geocoding service and requests driving, walking or other supported route profiles between the resolved coordinates. The response contains route geometry, distance and estimated duration for the main route and its alternatives. The module normalises the provider response into the internal route representation used by the rest of the system, so that scoring and selection do not depend on the provider format. Invalid or ambiguous locations are detected at this stage and reported to the user with an explanatory message rather than being passed forward silently. The Mapbox token is read from the server configuration.

6.2.2 Data Processing and Integration Module

The data processing module prepares the place, facility and accessibility data used by the system. It applies the cleaning and preprocessing operations described in Section 4.4: removal of duplicates, normalisation of names and categories, validation of coordinates, harmonisation of attribute values into the categories yes, no and UNKNOWN, and attachment of source and timestamp metadata. Records originating from external geographic data, from user reports, from verified entries and from synthetic or demonstration sources are kept distinguishable through a source field, and the interface can label them accordingly. The module never converts a missing value into a positive value. A missing attribute remains UNKNOWN and is later treated conservatively by the scoring module.

6.3 FEATURE PROCESSING

6.3.1 Accessibility Profile Module

The profile module stores the accessibility category chosen by the user: wheelchair or mobility assistance, limited mobility, visual accessibility, hearing accessibility, or general accessibility requirements. The profile is attached to the user record and is read by the scoring, route selection, place ranking and planner modules. The profile determines the weight vectors described in Section 4.9, and therefore the same set of routes or places can be ordered differently for different users. The profile can be changed at any time from the profile and settings pages of the dashboard.

6.3.2 Accessibility Scoring and Confidence Module

This module implements the algorithms of Section 4.9. For each place or route segment it loads the attribute values, selects the weight vector of the user profile, computes the score, the coverage and the upper bound, and builds the explanation that lists every factor with its

contribution. It also computes the attribute-level and place-level confidence from the source, verification state and age of each attribute. The score, the confidence level (HIGH, MEDIUM, LOW or UNKNOWN) and the explanation are returned together, so that the frontend never has to display a score without its supporting evidence.

6.4 ALGORITHM IMPLEMENTATION

6.4.1 Authentication and Access Control Module

The authentication module registers users, verifies credentials and issues a signed token that the frontend attaches to subsequent requests. Passwords are stored only in hashed form. Two roles are supported. An ordinary user can search routes and places, select a profile, submit reports, use the planner and chat with the assistant. A maintainer can, in addition, review reports, verify or reject accessibility data, and manage the data sources. Each protected endpoint declares the role it requires, and requests without a valid token or with an insufficient role are rejected with an appropriate HTTP status. Separating the two roles is important for data quality because verification, which raises data confidence, must not be available to every user.

6.4.2 Route Evaluation and Recommendation Module

The route module receives the normalised alternatives and computes, for each of them, the objective values: distance, travel time, accessibility, obstacle risk and route confidence. It then identifies the non-dominated routes by Pareto filtering and selects the recommended route from this set using the profile-specific weights. The complete set of alternatives is still returned, with the recommended route marked, so that users can compare them and choose differently. The module is deterministic: identical inputs give identical outputs, which simplifies testing.

6.4.3 Obstacle Reporting and Rerouting Module

Users submit reports that identify the type of problem (for example a broken ramp, a blocked path, an inaccessible entrance, an unavailable elevator or construction), the location and an optional description. A report is stored with its time and its author and, depending on its type and the verification state, is converted into an obstacle that contributes to the obstacle risk of the routes that pass near it. When a new obstacle affects the currently displayed route, the rerouting function re-evaluates the alternatives with the obstacle included, recalculates the scores and recommends a new suitable route. The behaviour is triggered by reports and by user requests; the prototype does not continuously sense obstacles.

6.4.4 Place and Facility Discovery Module

This module retrieves accessible hotels, restaurants, attractions and buildings for the selected location. Retrieval is location-specific: the query is built from the coordinates and the bounding area of the selected destination, so that results for one destination are never reused for another. Each result carries its accessibility attributes, confidence and source, and is ranked by the profile-weighted score. Results with many UNKNOWN attributes are not hidden but are presented with a clear indication, and verified results are distinguished from unverified ones.

6.4.5 AI Travel Planner Module

The planner collects the origin, destination, travel dates, number of travellers, accessibility profile and preferences. The backend retrieves the relevant places, facilities, accessibility attributes and route summaries for the destination, assembles them into a structured context, and calls Gemini with an instruction that restricts the model to the supplied entities. Gemini returns a day-wise itinerary, accessibility considerations and explanations, which the backend returns to the frontend. Because the context is location-specific, a plan for Chennai to Mahabalipuram uses different entities from a plan for Bengaluru to Mysuru. The planner is used for presentation and explanation; the accessibility facts it mentions come from the retrieved context, and a post-generation check of the output against the retrieved entities is listed as planned work.

6.4.6 Hybrid Chatbot Module

The chatbot first classifies each message. Messages that ask for facts about routes, facilities, accessibility attributes, datasets or locations are answered directly by backend functions that read the stored data. Messages that ask for explanation, summarisation, personalised suggestions or natural-language planning are passed to Gemini together with the structured context retrieved for that question. This division prevents the language model from inventing facts that must be exact, such as whether a hotel has an accessible room, while still allowing natural conversation.

6.4.7 Dashboard and Visualisation Module

The dashboard is organised into the sections Find Accessible Route, Live Map, Accessibility Score, Accessibility Overview, Accessible Categories, Recent Reports, Saved Places, Profile, Settings, Nearby Places, Accessible Buildings, Reports and Ratings, AI Travel Planner and Chatbot. Cards present places and routes, charts summarise accessibility attributes, route cards allow side-by-side comparison, the map displays route geometry and

markers, and hover details give a short explanation of a score or an attribute without leaving the page. The dashboard is described further in Section 6.6.

6.5 REFERENCE IMPLEMENTATION AND EXECUTION

To validate the algorithms independently of the web application, the formulas of Chapter 4 were implemented a second time as a short, self-contained Python program using NumPy: the accessibility score (Equations (4.1) to (4.3)), the noisy-OR confidence (Equation (4.4)), the route vector and normalisation (Equations (4.6) to (4.8)), Pareto dominance and utility (Equations (4.9) and (4.10)), the obstacle risk with a haversine test (Equation (4.11)) and the place ranking (Equation (4.12)). It does not call Mapbox or Gemini. The program generates the synthetic dataset of Section 4.3 and the synthetic route requests of Section 7.1.1, runs experiments E0 to E6, writes all numbers to one results file and draws the figures of Chapter 7. A fixed seed (42) makes every run reproducible. The run used Python 3.13.16 and NumPy 2.5.3.

6.6 USER INTERFACE

The dashboard presents the system in sections: Find Accessible Route, Live Map, Accessibility Score, Accessibility Overview, Accessible Categories, Recent Reports, Saved Places, Profile, Settings, Nearby Places, Accessible Buildings, Reports and Ratings, AI Travel Planner and Chatbot. User and maintainer roles see the functions that suit their role.

Interactive elements include the live map with route layers, route cards for visual route comparison, accessibility indicators and score gauges, overview charts, category cards, hover-based details that show feature breakdowns and confidence levels, and forms for reports and planning.

The system presents one recommended route together with the alternatives that remain on the Pareto front, explains the recommendation, and allows the user to save places, report obstacles and ask follow-up questions. The user keeps control; the system does not hide alternatives.

Clear labels, consistent colours with additional text or icon cues, concise explanations and visible confidence information are used so that the output can be interpreted without technical knowledge. A usability review forms part of the testing described in Section 7.7.

The screenshots of the running prototype are given in Figures 6.1, 6.2 and 6.3, each with a short description.

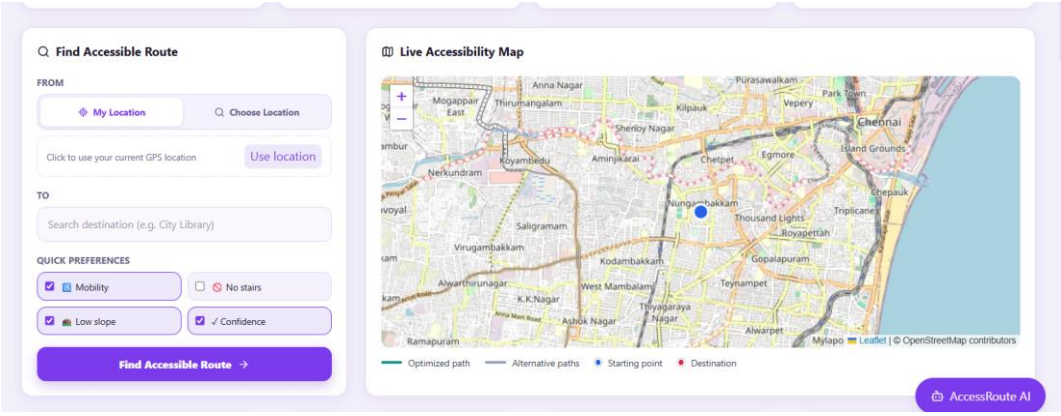


Figure 6.1 Find Accessible Route and Live Accessibility Map

The route search panel takes the start point (current GPS location or a chosen location) and a destination, and offers quick preferences such as Mobility, No stairs, Low slope and Confidence. The Live Accessibility Map beside it draws the optimised path and the alternative paths on OpenStreetMap, with markers for the starting point and the destination.

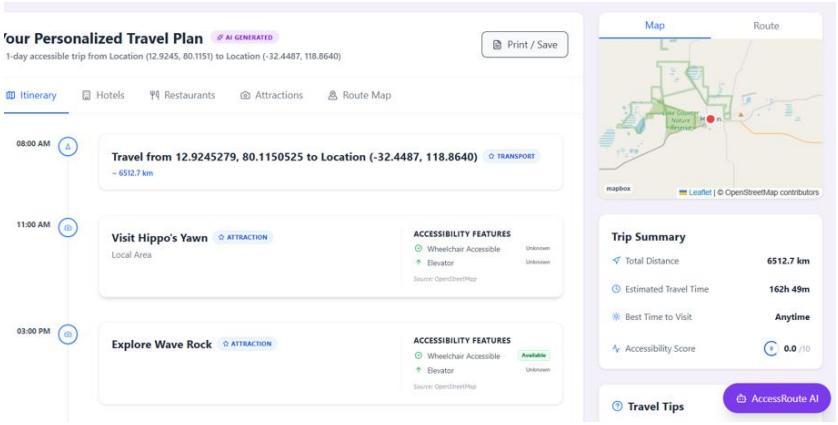


Figure 6.2 AI Travel Planner: Personalised Travel Plan

The AI Travel Planner produces a day-wise accessible itinerary with timed stops, transport legs and, for each place, accessibility features (wheelchair access, elevator) with their data source. The side panel shows the map, route, trip summary (distance, travel time, accessibility score) and travel tips.

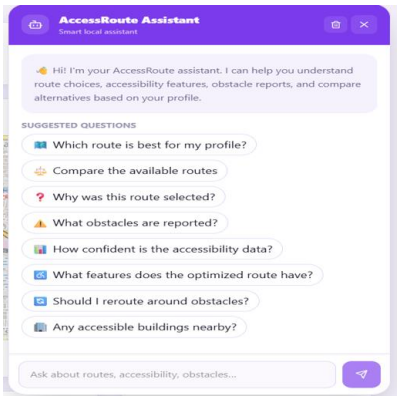


Figure 6.3 AccessRoute AI Hybrid Chatbot

The AccessRoute assistant answers questions about route choice, accessibility features, reported obstacles, data confidence and nearby accessible buildings. Suggested questions guide the user, and a text box accepts free-form queries.

CHAPTER 7
RESULTS AND DISCUSSION

This chapter reports the results of AccessRoute. Because AccessRoute is a decision-support system and not a trained predictive model, its results are the behaviour of its algorithms together with the testing of the application. The algorithm results were produced with a Python reference implementation of the formulas of Chapter 4 on synthetic data with a fixed random seed; they show how the algorithms behave and are not measurements of accessibility in the real world or of the Mapbox and Gemini integration.

7.1 EXPERIMENTAL SETUP

Experiments E0 to E6 (Section 4.8) were run with the reference implementation of Section 6.5; Section 4.3 describes the synthetic place dataset.

7.1.1 Synthetic Route Requests

The benchmark contains 3,000 route requests, 600 per profile, each with 3 to 6 candidate routes between the same origin and destination. The first candidate is the shortest; the others are up to 35 % longer, and the generator ties the share of accessible segments to the detour, so that longer routes are on average more accessible. Each route has 8 to 25 segments whose accessibility and confidence come from four segment features, each known with probability 0.7. For the wheelchair and limited-mobility profiles a route may contain a known blocking barrier, with a probability that rises with the number of segments and falls with route quality. Each request has 0 to 3 obstacles of low, medium or high severity (probabilities 0.4, 0.4, 0.2). These assumptions determine the size of every effect below; for example, because the generator rewards the detour, a distance-only baseline is expected to lose. The experiments therefore verify that the algorithms use the information correctly and show the shape of the trade-off; they do not estimate the gain for real travellers.

7.2 EVALUATION METRICS

The metrics are: passed over executed checks for functional correctness (Section 7.3.1); the maximum difference from the utilities printed in the worked example; the share of requests in which the recommendation differs from the shortest route, the mean accessibility gain in points and the mean extra distance in per cent, and the share of chosen routes that contain a known blocking barrier (Section 7.4.1); the share of recommendations that do not change under weight noise (Section 7.4.2); the overlap of the top-5 places and the Spearman rank

correlation between profiles (Section 7.5.1); the share of requests rerouted after a report on the recommended route (Section 7.5.2); and the run time against input size (Section 7.3.2).

7.3 ALGORITHM PERFORMANCE

7.3.1 Correctness and Property Validation

Experiment E1 executed 22 test cases that comprise 25,884 individual checks on the reference implementation, and all 25,884 passed (Table 7.1). The cases test invariants of the formulas (for example that the score lies between 0 and 100 and that replacing a known value by UNKNOWN never raises the score), the worked example of Chapter 4, and the dominance and normalisation properties on the 3,000 synthetic requests. Cases UT-06 and UT-12 to UT-14 reproduce the worked example of Section 4.9.1 and Tables 4.7 and 4.8; the largest difference between a computed utility and the printed value is 0.0005, which is the rounding of the printed values. The tests show that the implementation is internally consistent; they do not show that the weights are appropriate for real users.

Table 7.1 Property and Regression Tests of the Reference Implementation

Module	Test cases	Checks passed / executed
Accessibility scoring	UT-01 to UT-06 (6)	13,851 / 13,851
Data confidence	UT-07 to UT-11 (5)	8 / 8
Route evaluation	UT-12 to UT-18 (7)	12,021 / 12,021
Obstacle module	UT-19 to UT-22 (4)	4 / 4
Total	22	**25,884 / 25,884**

7.3.2 Run Time

Experiment E6 measured the cost of the algorithms. Scoring one place takes 8.2 µs and the confidence of one attribute 6.6 µs. Evaluating five routes of 20 segments with three obstacles takes 0.046 ms, and the Pareto filter with recommendation for five routes 0.060 ms. As Figure 7.1 shows, the pairwise Pareto filter grows quadratically, as predicted in Section 4.9.4: 100 routes take 10.9 ms and 500 routes 254 ms, which is irrelevant for the handful of alternatives that a routing service returns. Ranking 1,000 places takes 56 ms. These are the run times of the algorithms only; the response time of the application is dominated by the Mapbox and Gemini calls and has not been measured.

7.4 COMPARATIVE ANALYSIS

7.4.1 Recommended Route versus Distance-Only Baseline

Experiment E2 compares the route recommended by AccessRoute with the baseline of a conventional application, the shortest route, on the 3,000 synthetic requests. Table 7.2 gives the result for each profile and Figure 7.1 shows the main columns.

Table 7.2 Recommended Route versus Shortest Route on 3,000 Synthetic Requests

Profile	Mean Pareto size	Differs from shortest (%)	Mean access. gain (points)	Mean extra distance (%)	Shortest blocked (%)	Chosen blocked (%)
Wheelchair	3.94	86.5	31.8	17.7	30.7	1.5
Limited mobility	3.94	62.7	18.8	9.4	29.0	7.8
Visual	4.02	78.7	21.4	13.9	0.0	0.0
Hearing	4.06	63.2	15.7	9.2	0.0	0.0
General	4.05	46.0	8.2	4.5	0.0	0.0

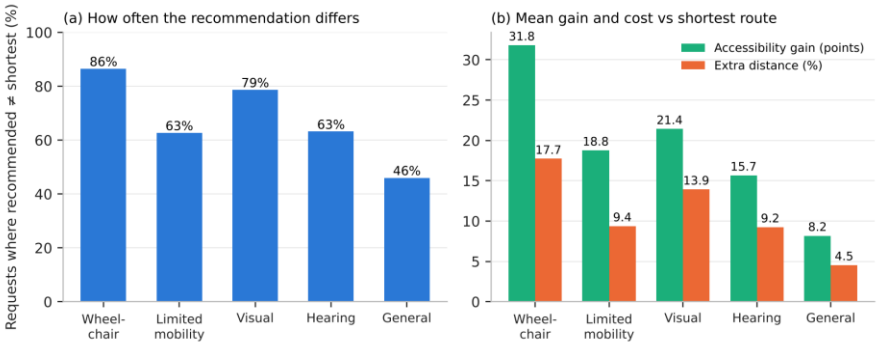


Figure 7.1 Difference between the Recommended and the Shortest Route: (a) Share of Requests, (b) Mean Gain and Cost

The recommendation differs from the shortest route in 86.5 % of the wheelchair requests and 78.7 % of the visual requests, against 46.0 % for the general profile. On average the recommended route is 31.8 accessibility points better for the wheelchair profile at 17.7 % additional distance (median 18.6 %), and 8.2 points for 4.5 % for the general profile. The trade-off is larger for the profiles for which accessibility matters more, as intended. For the wheelchair and limited-mobility profiles the shortest route contains a known blocking barrier in 31 % and 29 % of the requests, against 1.5 % and 7.8 % for the recommended route. The barrier rate of the shortest route follows from the generator assumption (Section 7.1.1); what the experiment shows is that the evaluation avoids the barrier whenever an alternative

without it exists. In contrast with a conventional application, which uses one objective and the same needs for every user, AccessRoute balances five objectives with profile weights, shows UNKNOWN data as such, explains its recommendation and reacts to user reports.

7.4.2 Sensitivity to the Weights

Experiment E3 perturbed the objective weights (Section 4.11). With noise of σ = 0.1 the recommendation stays the same in 94 % to 96 % of the cases across the profiles, and with σ = 0.2 in 87 % to 91 % (Figure 7.2(a)). The recommendation is therefore stable against small changes of the design weights and changes mainly in requests where two routes have nearly equal utility, where either would be a reasonable choice.

7.5 RECOMMENDATION RESULTS

7.5.1 Personalisation across Profiles

Experiment E4 ranked 300 random sets of 40 places with each of the five profiles. Figure 7.2(b) shows the average overlap of the top five places for each pair of profiles. The mean overlap over the ten pairs is 67 %, the mean Spearman rank correlation 0.86, and the profiles agree on the best place in 46 % of the sets. The visual profile differs most from the others (overlap 42 % with the wheelchair profile) because it weights obstacles and stairs most, and the hearing and general profiles are the most alike (85 %). Personalisation is thus real but not total: the profiles share the evidence and the weights change the order.

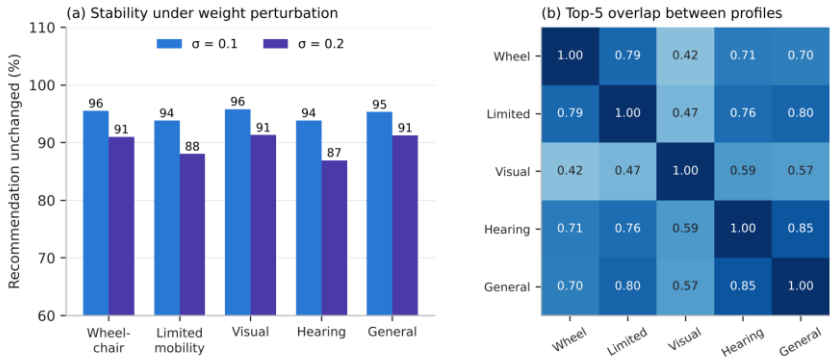


Figure 7.2 Weight Sensitivity and Personalisation: (a) Recommendations Unchanged under Weight Noise, (b) Top-5 Overlap between Profiles

7.5.2 Unknown Information and Rerouting

In the ranking of Section 4.9.7 a place whose must-have feature is UNKNOWN is listed in a separate unverified group and not hidden. For the wheelchair profile the must-have feature is the step-free entrance; among the places of a candidate set that are not excluded by a known violation, 55.7 % belong to the unverified group in the reference dataset. A system

that treats missing data as available would have presented them as accessible. Experiment E5 placed one fresh, high-severity obstacle on the recommended route of 2,000 requests and re-evaluated the alternatives (Algorithm 4.6). The recommendation changed in 74.9 % of the requests, with a mean utility improvement of 0.154; in 2.2 % the obstacle affected every alternative and the route was kept with a warning. In control cases with an obstacle on a route other than the recommended one, the recommendation was unchanged in 100 % of the cases. The rerouted share by profile is Wheelchair 77 %, Limited mobility 78 %, Visual 94 %, Hearing 74 %, General 51 %, lowest for the general profile (smallest obstacle weight) and highest for the visual profile (largest).

7.6 DISCUSSION

For travellers, AccessRoute offers routes and places ordered according to their own needs, with the evidence behind every score; for caregivers and planners it produces itineraries that respect the selected profile and state when information is unavailable; for maintainers, the verification role and source tracking improve data quality over time. The algorithm results come from synthetic data and a reference implementation. They show that the formulas are implemented consistently, that the recommendation responds to the profile, to missing data and to obstacles in the intended direction, that it is stable against small changes of the weights, and that its cost is small. They do not show how accessible the recommended routes are in a real city, how large the gain is for real travellers, or how reliable the open, user and maintainer data are; the size of every effect depends on the generator assumptions of Section 7.1.1 and Appendix B. The design weights are chosen by the team and not learned from data, the generator rewards a detour with better accessibility (which favours the recommendation over the baseline by construction), the Gemini components were not evaluated, and results from one region may not generalise to another. A study with real travellers and verified routes is needed for those claims.

7.7 SOFTWARE TESTING AND VALIDATION

Testing of the application is organised from small units, such as the scoring function, to complete scenarios. The unit-level checks of the algorithms were executed on the reference implementation (Section 7.3.1). Defects found are recorded in the project defect log.

CHAPTER 8
CONCLUSION AND FUTURE ENHANCEMENT

8.1 SUMMARY OF THE WORK

This project designed and developed AccessRoute, an AI-powered accessibility-aware navigation and travel planning system. The work was motivated by the observation that conventional navigation applications optimise distance or time and assume that all travellers have the same needs, so that the fastest route can be unusable for a wheelchair user, a person with limited mobility, or a person with visual or hearing impairment. AccessRoute addresses this gap by treating accessibility, uncertainty and user reports as first-class inputs to routing and travel planning.

8.2 KEY CONTRIBUTIONS

Personalised accessibility profiles make the system adapt its weights and recommendations to the user. A transparent accessibility score with a factor breakdown explains why a place or a route is considered suitable. Data confidence and source tracking separate verified information from unverified or unknown information, and unknown attributes are never assumed to be available. Multi-objective route comparison and Pareto-based selection allow the system to present non-dominated alternatives and recommend one according to the profile. User obstacle reporting supports obstacle-aware evaluation and report-triggered rerouting. Accessibility-aware place discovery ranks hotels, restaurants, attractions and buildings by profile-weighted scores. The AI Travel Planner and the hybrid chatbot use Gemini on structured, location-specific context retrieved by the FastAPI backend, while factual accessibility information is provided deterministically.

8.3 LIMITATIONS

Table 8.1 Limitations of the Present Work

Limitation	Explanation
Incomplete accessibility information	Many places have no recorded ramp, lift, restroom or parking information; such values remain UNKNOWN
Dependence on external APIs	Geocoding, routing and AI generation depend on Mapbox and Gemini; outages, quota limits and policy changes affect the system
Varying data quality	Sources differ in accuracy, currency and completeness
Coverage	Data coverage differs between regions, and results are limited where data is sparse
Dynamic obstacle information	Obstacles are known only if reported; continuous live sensing is not implemented
Network dependence	The system requires connectivity; offline operation is not supported
Third-party data limitations	External place and accessibility data may be outdated or inconsistent
AI hallucination risk	Language models can produce unsupported statements; the design reduces but does not eliminate this risk until output validation is implemented
Need for human verification	Accessibility information must be verified by people, and the system cannot replace on-site verification
Design weights	Weights are chosen by design and not learned from data
Evaluation	Algorithm-level results come from synthetic data; the application test cases and a user study with real travellers are not completed

8.4 FUTURE ENHANCEMENTS

Larger accessibility datasets, a crowd-sourced verification workflow with maintainer review, improved handling of report ageing, and expanded geographic coverage would raise the coverage and confidence of the data. A stronger accessibility ontology would give attributes a consistent meaning across sources.

Execution of the full test suite, measurement of the response time of the running application, and a controlled user study with real travellers and verified routes would provide the evidence that the synthetic evaluation of this report cannot.

REFERENCES

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, no. 1, pp. 269-271, 1959.

[2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," IEEE Trans. Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, 1968.

[3] World Health Organization and World Bank, World Report on Disability. Geneva, Switzerland: WHO, 2011.

[4] World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[5] OpenStreetMap contributors, "OpenStreetMap Wiki." [Online]. Available: https://wiki.openstreetmap.org.

[6] Wheelmap.org, Sozialhelden e.V. [Online]. Available: https://wheelmap.org.

[7] AccessMap, Taskar Center for Accessible Technology, University of Washington. [Online]. Available: https://www.accessmap.io

[8] A. D. Sobek and H. J. Miller, "U-Access: A web-based system for routing pedestrians of differing abilities," Journal of Geographical Systems, vol. 8, no. 3, pp. 269-287, 2006.

[9] M. Saha et al., "Project Sidewalk: A web-based crowdsourcing tool for collecting sidewalk accessibility data at scale," in Proc. CHI Conf. on Human Factors in Computing Systems, 2019.

[10] Ministry of Statistics and Programme Implementation, Government of India, "Disabled Persons in India: A Statistical Profile 2016," 2016.

[11] Government of India, "The Rights of Persons with Disabilities Act, 2016," The Gazette of India, 2016.

[12] Department of Empowerment of Persons with Disabilities, Government of India, "Accessible India Campaign (Sugamya Bharat Abhiyan)." [Online]. Available: https://disabilityaffairs.gov.in

[13] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, Art. no. 248, 2023.

[14] H. Wickham, "Tidy data," Journal of Statistical Software, vol. 59, no. 10, pp. 1-23, 2014.

[15] R. Y. Wang and D. M. Strong, "Beyond accuracy: What data quality means to data consumers," Journal of Management Information Systems, vol. 12, no. 4, pp. 5-33, 1996.

[16] C.-L. Hwang and K. Yoon, Multiple Attribute Decision Making: Methods and Applications. Berlin, Germany: Springer-Verlag, 1981.

[17] T. L. Saaty, The Analytic Hierarchy Process. New York, NY, USA: McGraw-Hill, 1980.

[18] M. Ehrgott, Multicriteria Optimization, 2nd ed. Berlin, Germany: Springer, 2005.

[19] K. Miettinen, Nonlinear Multiobjective Optimization. Boston, MA, USA: Kluwer Academic Publishers, 1999.

[20] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020.

[21] Google, "Gemini API documentation." [Online]. Available: https://ai.google.dev/gemini-api/docs.

[22] M. Haklay and P. Weber, "OpenStreetMap: User-generated street maps," IEEE Pervasive Computing, vol. 7, no. 4, pp. 12-18, 2008.

[23] OpenStreetMap Foundation, "Copyright and license." [Online]. Available: https://www.openstreetmap.org/copyright.

[24] H. T. Kung, F. Luccio, and F. P. Preparata, "On finding the maxima of a set of vectors," Journal of the ACM, vol. 22, no. 4, pp. 469-476, 1975.

[25] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," IEEE Trans. Evolutionary Computation, vol. 6, no. 2, pp. 182-197, 2002.

[26] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conf., 2010, pp. 56-61.

[27] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357-362, 2020.

[28] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science and Engineering, vol. 9, no. 3, pp. 90-95, 2007.

[29] R. W. Sinnott, "Virtues of the haversine," Sky and Telescope, vol. 68, no. 2, p. 159, 1984.

[30] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com

[31] Pydantic, "Pydantic documentation." [Online]. Available: https://docs.pydantic.dev.

[32] Mapbox, "Mapbox documentation: Geocoding, Directions and Maps." [Online]. Available: https://docs.mapbox.com.

[33] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, CA, USA, 2000.

[34] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[35] J. W. Tukey, Exploratory Data Analysis. Reading, MA, USA: Addison-Wesley, 1977.

APPENDIX

APPENDIX A: DATA DICTIONARY OF ACCESSIBILITY ATTRIBUTES

Table A.1 Accessibility Attributes and Their Value Domains

Attribute	Meaning	Allowed Values	Typical Use
Step-free entrance	Entrance reachable without steps	yes, no, UNKNOWN	Wheelchair and limited mobility profiles
Ramp	Ramp available at an entrance or level change	yes, no, UNKNOWN	Wheelchair and limited mobility profiles
Elevator	Elevator available	yes, no, UNKNOWN	Multi-storey buildings
Ground-floor access	Main services on the ground floor	yes, no, UNKNOWN	Limited mobility profile
Accessible restroom	Restroom suitable for wheelchair users	yes, no, UNKNOWN	All mobility profiles
Accessible parking	Reserved accessible parking	yes, no, UNKNOWN	Mobility profiles
Accessible room	Accessible hotel room	yes, no, UNKNOWN	Hotel ranking
Stairs	Stairs on the route or at the entrance	yes, no, UNKNOWN	Route and place evaluation
Slope	Steep slope on the route	yes, no, UNKNOWN	Route evaluation
Obstacle	Reported obstacle near the place or route	none, active, resolved	Obstacle risk
Signage and information	Clear signs or information support	yes, no, UNKNOWN	Visual and hearing profiles

APPENDIX B: EXPERIMENT SETTINGS AND CODE LISTINGS

The synthetic generator used for the experiments of Chapter 7 draws all random numbers with the seed 42; the experiment program and its results file are provided with the report.

PO & PSO ATTAINMENT

The program outcomes and program specific outcomes addressed by the project are listed below with a justification based on the work reported.

Table PO.1 Program Outcome Attainment

PO No	Graduate Attribute	Attained	Justification
PO1	Engineering knowledge	Yes	Applying data science, algorithm design and web engineering knowledge to build an accessibility-aware navigation and travel planning system.
PO2	Problem analysis	Yes	Analysing why conventional navigation fails users with different accessibility needs, and identifying data uncertainty as a central difficulty.
PO3	Design/Development of solutions	Yes	Designing the scoring, confidence, Pareto selection, obstacle-aware rerouting and hybrid AI components of AccessRoute.
PO4	Conduct investigations of complex problems	Yes	Studying related work, formulating the multi-objective problem and designing test cases. Quantitative investigation is carried out on a synthetic reference implementation.
PO5	Engineering Tool Usage	Yes	Using FastAPI, Mapbox, the Gemini API, Python data libraries and version control.
PO6	The Engineer and The World	Yes	Addressing the mobility and information needs of people with disabilities through a transparent decision-support tool.
PO7	Ethics	Yes	Representing missing data as UNKNOWN, separating real from synthetic data, avoiding unsupported claims and protecting keys and user data.
PO8	Individual and Team work	Yes	Dividing backend and frontend responsibilities between the two team members and integrating them.
PO9	Communication	Yes	Preparing documentation, diagrams and a dashboard that communicate scores and recommendations clearly.
PO10	Project Management and Finance	Yes	Planning the work in incremental stages and using freely available tools and services within their usage limits.
PO11	Life-long learning	Yes	Learning new frameworks, geographic services and generative AI practices during the project.

Table PO.2 Program Specific Outcome Attainment

PSO No	Graduate Attribute	Attained	Justification
PSO1	Ability to design and implement intelligent systems using AI, ML and Data Science techniques	Yes	Designing a data-driven recommendation system with scoring, multi-objective selection and AI-assisted planning.
PSO2	Ability to handle real-world data using appropriate tools, programming skills and analytical methods	Yes	Collecting, cleaning, integrating and tracking the provenance of geographic and accessibility data with Python.